



GitHub Trending Top 10

每日热门开源项目深度解读

2026-08-20

今日榜单

1	harry0703/MoneyPrinterTurbo	Python	利用 AI 大模型和自动化工作流，根据主题或关键词一键生成高清短视频。Generate HD short vi...	3天
2	volcengine/OpenViking	Python	Self-evolving Context Database for AI Agents. Unify Agent Memory, Knowledge RAG ...	2天
3	chaitanyagiri/munder-difflin	TypeScript	local multi-agent harness	2天
4	mukul975/Anthropic-Cybersecurity-Skills	Python	817 structured cybersecurity skills for AI agents • Mapped to 6 frameworks: MITR...	3天
5	akitaonrails/ai-memory	Rust	Solution for long term memory for agent coding CLIs and to facilitate handoff be...	3天
6	nautechsystems/nautilus_trader	Rust	Production-grade Rust-native trading engine with deterministic event-driven arch...	NEW
7	mattpocock/skills	Shell	Skills for Real Engineers. Straight from my .agents directory.	NEW
8	obra/superpowers	Shell	An agentic skills framework & software development methodology that works.	NEW
9	jundot/omlx	Python	LLM inference server with continuous batching & SSD caching for Apple Silicon — ...	3天
10	santifer/career-ops	JavaScript	Open-source AI job search: scan job portals, evaluate listings with a structured...	NEW

#1

Python

连续 3 天

MoneyPrinterTurbo

利用 AI 大模型和自动化 workflow，根据主题或关键词一键生成高清短视频。Generate HD short videos from a topic or keyword with an automated AI workflow.

Stars **112,238** Forks **17,000** Today **+2,221**

<https://github.com/harry0703/MoneyPrinterTurbo>

项目简介：MoneyPrinterTurbo 是一站式 AI 短视频生成工具，用户只需输入一个主题或关键词，它就能自动完成从脚本撰写、素材匹配、字幕生成到背景音乐合成和视频渲染的全流程，最终输出一条高清短视频。它解决的是内容创作者在短视频制作中耗时耗力的痛点，将原本需要数小时的制作流程压缩到几分钟。

核心功能：第一，AI 驱动的全自动 workflow，串联大模型、素材库、语音合成和视频渲染引擎；第二，支持 WebUI 和 API 两种交互模式，既适合个人点击操作，也方便开发者集成到自己的系统中；第三，多语言支持，项目提供中、英、日等多语言界面，并适配不同大模型供应商（如 Kimi、火山引擎等）；第四，灵活的配置选项，用户可自定义视频比例、配音音色、字幕样式等细节参数。

适用场景：最典型的使用者是短视频自媒体运营者，可以用它批量生产口播类、科普类或资讯类视频；其次是营销人员，能快速生成产品宣传素材；此外，教育领域从业者也可以用它制作课程讲解短视频。对于缺乏视频剪辑技能但需要持续产出内容的个人或小组，这个工具能显著降低创作门槛。

技术亮点：项目采用模块化流水线架构，将“文案生成—素材检索—语音合成—视频合成”解耦为独立环节，每个环节都可替换底层服务（如不同大模型或素材库），这种设计保证了灵活性和可扩展性。另外，它通过 API 方式调用大模型来提炼搜索关键词，让 AI 不仅写文案，还参与画面决策，提升了素材匹配的精准度。项目本身基于 Python 构建，跨平台支持 Windows、macOS 和 Linux，部署门槛较低。

上手建议：新用户应先查看 README 中的安装指引，推荐使用 Docker 方式快速启动，避免环境配置问题。使用前需准备大模型 API Key 和素材库访问凭证，建议先用自带的示例主题跑通流程，再逐步调整参数。对于想贡献代码的开发者，可以从 GitHub Issues 中挑选标注为“good first issue”的任务入手，熟悉项目的模块划分后再深入核心逻辑。

#2

Python

连续 2 天

OpenViking

Self-evolving Context Database for AI Agents. Unify Agent Memory, Knowledge RAG and Skills.

Stars **30,708** Forks **2,371** Today **+804**

<https://github.com/volcengine/OpenViking>

项目简介：OpenViking 是火山引擎开源的一个面向 AI Agent 的“上下文数据库”。它将 Agent 的记忆、知识与技能统一映射为一个虚拟文件系统（viking:// 协议），让 Agent 像开发者操作文件一样，用 ls、tree、find 等命令来浏览和管理自己的上下文，而不是去查询一个黑盒向量数据库。其核心目标是解决 AI Agent 在长期运行中上下文管理混乱、检索不可控、Token 消耗大的问题。

核心功能：一是**统一上下文模型**，将记忆、资源和技能都抽象为带 URI 的文件，实现确定性定位与操作；二是**三级分层存储（L0/L1/L2）**，写入时自动生成摘要、概览和详情，任务只需加载所需的深度，大幅节省 Token；三是**目录递归检索**，先定位高相关目录，再逐层下钻，保证返回结果自带完整上下文；四是**可观测检索**，每次查询都保留浏览轨迹，可追溯错误结果来源；五是**会话沉淀为长期记忆**，会话结束后异步提取用户偏好与经验，存入长期记忆。

适用场景：主要面向构建复杂 AI Agent 的开发者与团队，尤其是需要长期记忆、多轮对话、知识库问答或技能调用的场景，如个人 AI 助手、企业知识管理、自动化 workflows 等。凡是希望 Agent 的上下文“可理解、可调试、可持久化”的项目，都能从中受益。其浏览器版 Studio 演示也降低了上手门槛，适合快速验证。

技术亮点：最值得关注的是“**文件系统即上下文**”的设计范式，它摒弃了传统向量库的“黑盒”检索，让上下文管理变得透明、可解释，且天然支持层级结构与权限隔离。三级分层加载是工程上的务实创新，直击 Token 成本痛点。此外，检索轨迹的可观测性为调试 Agent 行为提供了极大便利，这在同类项目中较为少见。

上手建议：建议从阅读官方文档的“快速开始”和“架构”章节入手，先了解核心概念（URI、上下文类型、分层机制）。随后可打开浏览器版 Studio 进行体验，直观感受其交互方式。若要集成到自己的 Agent，可参考官方示例代码，从创建资源目录和写入记忆开始，逐步尝试分层检索与会话沉淀功能。若想贡献代码，可先浏览 GitHub Issues 中的 good first issue 标签，并熟悉项目的 Python 代码结构与 AGPLv3 许可证。

#3

TypeScript

连续 2 天

munder-difflin

local multi-agent harness

Stars **2,924** Forks **337** Today **+795**<https://github.com/chaitanyagiri/munder-difflin>

项目简介：Munder Difflin 是一个开源的本地多智能体协调框架，它把你已经拥有的终端 AI 编程工具（如 Claude Code、Codex、Grok 等）包装成一个个独立"员工"，并在你的电脑上模拟出一个虚拟办公室。它的核心价值在于，让你不在电脑前时，这些 AI 代理能像你的克隆体一样持续工作，并相互协作完成复杂任务。

核心功能：1) 多引擎适配——无缝集成10余种主流终端 AI CLI，支持自带密钥或本地模型；2) 智能协调中枢——通过"GOD agent"（即你的克隆体 Michael）负责任务分配、路由和升级决策；3) 持久记忆系统——基于 Markdown 的快速语义记忆层，让代理跨会话记住上下文；4) 可视化办公界面——用 Pixi.js 渲染的 2D 办公室，代理以头像形式走动、收发消息，交互直观有趣。

适用场景：适合需要并行处理多个编码任务的专业开发者或小团队，尤其是当你需要"分身"同时处理代码审查、Bug 修复、文档编写等不同类型工作时。也适合喜欢探索 AI 多代理协作模式的技术爱好者，想在不增加订阅成本的前提下，充分利用现有 AI 工具的 API 额度。

技术亮点：项目采用 Electron + React + TypeScript 构建桌面应用，底层用 node-pty 真实模拟终端进程，确保与各 AI CLI 的字节级兼容。架构上采用"黑板模式"（blackboard）实现代理间信息共享，配合"邮箱-路由"机制解耦任务分发，而 GOD agent 作为监督者形成清晰的层级控制，这种设计兼顾了分布式协作与集中管控。

上手建议：项目目前处于原型阶段（v0.4.4），但安装使用很直接——先确保你已安装至少一个支持的终端 AI CLI（如 Claude Code），然后通过 npm 或源码构建启动应用即可。如果想贡献代码，建议先阅读 README 中的架构文档和项目结构说明，从 GitHub Issues 中标记"good first issue"的任务入手，社区 Discord 也是获取帮助的好渠道。

#4

Python

连续 3 天

Anthropic-Cybersecurity-Skills

817 structured cybersecurity skills for AI agents · Mapped to 6 frameworks: MITRE ATT&CK, NIST CSF 2.0, MITRE ATLAS, D3FEND, NIST AI RMF & MITRE F3 (Fight Fraud) · agentskills.io standard · Works with Claude Code, GitHub Copilot, Codex CLI, Cursor, Gemini CLI & 20+ platforms · 29 security domains · Apache 2.0

Stars **30,170** Forks **3,580** Today **+766**

<https://github.com/mukul975/Anthropic-Cybersecurity-Skills>

项目简介：这是一个社区创建的开源项目，为 AI 智能体提供 817 个结构化网络安全技能，覆盖 29 个安全领域。它解决的核心问题是让 AI 助手具备资深安全分析师的专业能力，在安全调查、威胁检测等任务中提供专家级指导。

核心功能：首先，技能库覆盖 29 个安全域，从数字取证到云安全、AI 安全等；其次，每个技能都映射到 6 个行业框架（MITRE ATT&CK、NIST CSF 2.0 等），方便按框架检索；第三，遵循 agentskills.io 开放标准，技能格式统一；第四，兼容 26+ 平台，包括 Claude Code、GitHub Copilot、Cursor 等主流 AI 编程工具。

适用场景：安全分析师可以用它加速威胁调查和事件响应，红队/蓝队成员可获取规范的攻防技术参考，AI 应用开发者可为其智能体注入专业安全能力。特别适合需要处理云安全事件、内存取证、AI 系统防护等复杂任务的场景。

技术亮点：项目最值得关注的是框架映射的精准性——每个技能并非机械地映射到所有框架，而是根据技能类型匹配相关框架（如取证技能映射 ATT&CK+CSF，AI 安全技能额外叠加 ATLAS 和 AI RMF）。项目采用 Python 实现，遵循 Apache 2.0 开源协议，社区活跃度高。

上手建议：直接克隆仓库，将技能库指向你的 AI 智能体即可开始使用。如果想贡献，可以从 CONTRIBUTING.md 入手，按照 agentskills.io 标准补充新技能或改进现有技能的框架映射。注意遵守 SECURITY.md 中的合法使用规范，仅对授权系统使用攻防类技能。

#5

Rust

连续 3 天

ai-memory

Solution for long term memory for agent coding CLIs and to facilitate handoff between different agent vendors

Stars **3,346** Forks **272** Today **+606**

<https://github.com/akitaonrails/ai-memory>

项目简介：ai-memory 是一个为 AI 编程代理提供长期记忆的 Rust 工具，解决跨会话、跨代理工具切换时上下文丢失的问题。它让用户可以在 Claude Code 中开始任务，中途退出后切换到 OpenAI Codex 继续，无需重复解释架构决策或遗留问题。

核心功能：第一，通过 MCP 配置和生命周期钩子自动捕获工作会话中的关键信息，涵盖主流代理工具（Claude Code、Codex、Cursor、Gemini CLI 等）；第二，会话结束时可生成摘要与交接文档，便于后续代理快速接管；第三，提供 ai-memory run 管理式工作流，实现跨多个代理工具的透明连续性；第四，支持细粒度捕获排除规则，可忽略敏感或无关内容。

适用场景：频繁在多个 AI 编码代理之间切换的开发者；需要在长时间、多阶段编码任务中保持上下文连贯性的团队；以及使用 CLI 代理进行复杂架构设计、需要记录失败尝试和未决问题的个人开发者。

技术亮点：采用 Rust 实现，性能优越且便于分发原生二进制；通过 MCP（Model Context Protocol）标准与各代理工具集成，而非针对单一工具做硬编码；支持多平台（Linux、macOS、Windows/WSL2），并针对不同代理的钩子机制做了细粒度适配。

上手建议：从安装对应平台的二进制或 Docker 镜像开始，按 README 中的支持矩阵找到你正在使用的代理工具（如 Claude Code 或 Codex），运行对应的 MCP 安装命令即可。想贡献代码可先从 docs/ 目录了解各代理的集成细节，或从 GitHub Issues 中挑选标注为 good first issue 的任务。

#6

Rust

NEW

nautilus_trader

Production-grade Rust-native trading engine with deterministic event-driven architecture

Stars **26,622** Forks **3,431** Today **+80**

https://github.com/nautechsystems/nautilus_trader

项目简介：

NautilusTrader 是一个生产级的开源交易引擎，用 Rust 编写核心，以 Python 作为策略层控制面。它解决了量化交易中“研究环境与实盘环境行为不一致”的核心痛点，让同一套策略代码无需修改即可从回测平滑过渡到实盘。

核心功能：

- **多资产多交易所支持**：涵盖加密货币（CEX/DEX）、外汇、股票、期货、期权甚至博彩交易所，通过模块化适配器接入任何有 REST/WebSocket 接口的场所。
- **确定性事件驱动架构**：回测与实盘共用同一套时间模型和执行语义，保证研究结果可复现、可平移。
- **Rust 核心 + Python 控制平面**：策略逻辑用 Python 编写，引擎性能由 Rust 保证，兼顾开发效率与运行速度。
- **纯 Rust 部署选项**：对性能要求极高的关键任务，也可完全用 Rust 编写交易系统。

适用场景：

主要面向量化基金、自营交易团队和独立开发者，用于构建高频或中低频交易系统。适合需要高吞吐、低延迟，同时又希望策略开发保持灵活性的团队，尤其是那些受够了回测与实盘行为不一致问题的用户。

技术亮点：

Rust 提供的类型安全和线程安全从根本上消除了内存错误和数据竞争，配合 mimalloc 分配器和 tokio 异步网络栈，在保证低延迟的同时维持了极高的可靠性。这种“编译型引擎 + 脚本化策略”的架构设计，在交易系统领域是一种兼顾性能与开发体验的优雅解法。

上手建议：

从官方文档（nautilustrader.io/docs）的快速入门开始，先跑通一个简单的回测示例。想深入贡献的

话，可以从 GitHub 上的 develop 分支入手，先浏览 `nautilus_core` 的 Rust 代码结构，或从测试用例和 issue 中找小任务练手。

#7

Shell

NEW

skills

Skills for Real Engineers. Straight from my .agents directory.

Stars **224,616** Forks **19,305** Today **+1,894**

<https://github.com/mattpocock/skills>

项目简介：这是 TypeScript 专家 Matt Pocock 开源的 AI 编程助手技能集，来自他日常真实工程实践而非“vibe coding”。它解决的核心问题是：AI 编码代理（如 Claude Code、Codex）常常误解需求、输出冗长、缺乏工程纪律，通过一系列可组合的“技能”文件来对齐人机意图、规范代理行为。

核心功能：① /grill-me 与 /grill-with-docs——让代理在动手前像资深工程师一样“拷问”需求，消除理解偏差；② /triage——利用 issue 标签自动分类任务；③ 与 GitHub/Linear/本地文件三种 issue 追踪器集成；④ 可组合的小型技能文件，按需安装、自由修改；⑤ 支持插件（只读自动更新）与源码复制（可编辑）两种安装哲学。

适用场景：正在用 Claude Code、Codex 等 AI 编码代理做真实项目开发的工程师；团队想建立统一的 AI 协作规范；对“代理失控”（答非所问、过度输出）感到头疼的开发者。特别适合需要严谨需求对齐的中大型工程，而非快速原型。

技术亮点：设计哲学是“小而美、可组合”——每个技能是独立的 Markdown 文件，不搞重流程框架（对比 GSD、BMAD），让工程师保留控制权。双通道分发（Claude Code 插件 vs skills.sh 复制）兼顾“订阅更新”与“fork 改造”两种社区文化，且通过 ADR 记录架构决策，工程味十足。

上手建议：30 秒安装：Claude Code 用户直接运行 `claude plugins install mattpocock-skills`，其他代理用 `npx skills@latest add mattpocock/skills`。装完后在仓库里跑一次 `setup-matt-pocock-skills` 做初始化配置，然后从 /grill-me 开始体验。想贡献的话，直接 fork 仓库改 SKILL.md 文件即可，项目鼓励“hack around and make them your own”。

#8

Shell

NEW

superpowers

An agentic skills framework & software development methodology that works.

Stars **274,598** Forks **24,577** Today **+557**

<https://github.com/obra/superpowers>

项目简介：Superpowers 是一套面向 AI 编程代理的软件开发方法论，通过可组合的技能模块和初始指令，让编码代理在开发流程中自动遵循从需求澄清、设计评审到测试驱动开发的完整规范。它解决的核心问题是：AI 编程工具往往过于“急于写代码”，缺乏系统性的工程流程约束，导致产出质量不稳定。

核心功能：一是“先问再做”的交互机制，代理启动后会主动澄清真实需求而非直接编码；二是将需求拆解为可读的规格说明和实现计划，确保设计经过用户确认；三是采用子代理驱动的开发模式，由多个代理分任务执行工程步骤并相互审查；四是内置 TDD、YAGNI、DRY 等工程原则，强调真正的红绿测试循环和最小化实现。

适用场景：适用于使用 Claude Code、Cursor、Codex、Gemini CLI 等主流 AI 编程工具的个人开发者或团队，尤其是那些希望让 AI 代理能够自主完成数小时级复杂开发任务、同时保持代码质量和项目可控性的场景。对于需要标准化 AI 辅助开发流程的企业团队，该项目也提供了可复用的方法论基础。

技术亮点：其核心创新在于将“技能”（skills）设计为可组合的模块化单元，并通过会话启动钩子（session-start hooks）实现自动触发，无需用户手动干预。项目同时支持超过 14 种主流 AI 编程工具的插件市场集成，展现了高度的生态兼容性，且通过“子代理驱动开发”模式实现了多代理协作的并行工程能力。

上手建议：如果你是使用者，可根据自己使用的工具（如 Claude Code 或 Cursor）按照 README 中的安装命令直接安装插件，无需额外配置即可体验完整流程。若想深入了解或贡献，建议先阅读项目的“What's Inside”部分了解技能模块结构，再通过 GitHub Issues 或社区讨论参与改进，注意项目支持独立为每个工具安装。

#9

Python

连续 3 天

omlx

LLM inference server with continuous batching & SSD caching for Apple Silicon — managed from the macOS menu bar

Stars **19,978** Forks **1,701** Today **+472**

<https://github.com/jundot/omlx>

项目简介：oMLX 是一个专为 Apple Silicon Mac 设计的本地 LLM 推理服务器，通过菜单栏应用即可管理模型加载、切换和推理服务。它解决了本地运行大模型时“既要便捷又要控制”的矛盾，让开发者无需命令行也能高效驾驭本地大模型。

核心功能：1) 连续批处理（Continuous Batching）提升 GPU 利用率；2) 分层 KV 缓存——热数据驻留内存、冷数据落盘 SSD，即使对话上下文变化也能复用历史缓存；3) 菜单栏图形化管理，支持一键固定常驻模型、按需自动换入重型模型、设置上下文长度；4) 提供 CLI 和 MCP 支持，可无缝对接 Claude Code 等编码工具。

适用场景：主要面向在 Mac 上进行本地 AI 开发的技术人员，尤其是使用 Claude Code、OpenCode 等 AI 编程工具的用户。适合需要长时间会话、频繁切换模型、且对数据隐私有要求的场景，例如代码审查、长文档分析或离线开发环境。

技术亮点：其“分层 KV 缓存”设计是核心创新——将高频访问的上下文保留在内存，低频历史写入 SSD，突破了传统本地推理的上下文长度限制。此外，针对 GLM-5.2 等模型提供自研 Metal 融合内核，实测 prefill 速度提升约 30 倍（845 vs 29 tok/s），展现了深度硬件优化的功力。

上手建议：最简路径是直接下载 `DMG` 安装包，按欢迎向导三步完成配置。技术用户可通过 Homebrew 安装并运行 `omlx start` 启动后台服务。若想贡献代码，建议先阅读 README 中的自定义内核构建说明——默认 pip 安装不会编译 Metal 内核，需完整 Xcode 环境才能发挥全部性能。

#10

JavaScript

NEW

career-ops

Open-source AI job search: scan job portals, evaluate listings with a structured A-F rubric into a 1.0-5.0 score, tailor your CV, track applications — runs locally in your AI coding CLI (Claude Code, Codex, OpenCode, Antigravity...)

Stars **66,157** Forks **12,776** Today **+198**

<https://github.com/santifer/career-ops>

项目简介：career-ops 是一个开源的 AI 求职助手，它把求职流程从“海投简历”变成“精准筛选”。项目通过 AI 代理自动扫描招聘网站，用结构化的 A-F 评分标准评估职位与你的匹配度，并生成定制化简历，全程在本地 AI 编程终端（如 Claude Code）中运行。它解决的是求职者被企业 AI 筛选系统“单向过滤”的不对称问题，让你用 AI 反向选择公司。

核心功能：1) **职位智能扫描与评分：**自动抓取招聘信息，按 1.0-5.0 分制评估职位匹配度，帮你快速过滤垃圾职位；2) **简历自动定制：**根据目标职位要求，自动调整简历关键词和项目经历，生成针对性版本；3) **申请流程追踪：**管理所有投递记录、面试进度和后续跟进；4) **本地化运行：**所有数据处理都在本地 CLI 环境中完成，保护个人隐私；5) **多平台兼容：**支持 Claude Code、Codex、OpenCode 等多种 AI 编程终端。

适用场景：适合正在积极求职的开发者、产品经理等技术人员，尤其是那些每天要浏览大量招聘信息、需要为不同职位反复修改简历的人。也适合希望将求职流程自动化的“量化求职者”——项目作者本人就是用这个系统评估了 740+ 职位，生成了 100+ 份简历，最终拿到理想工作。需要注意的是，它需要一定的命令行使用基础。

技术亮点：项目将“Agent Skills”标准落地到求职场景，采用多代理协作架构，让不同 AI 代理分别负责扫描、评估、写简历等任务。它不依赖单一 AI 平台，而是通过 Agent Skill 标准适配多种 CLI 环境，技术栈上结合了 Node.js 和 Go，并使用 Playwright 进行网页自动化抓取。这种“AI 原生工作流”的设计思路，代表了对传统求职工具的重构。

上手建议：如果你是使用者，先阅读 README 中的安装指南，在本地配置好支持的 AI CLI 环境（如 Claude Code），然后按照文档初始化项目并输入你的简历和目标职位关键词。如果你是贡献者，可以从 GitHub Issues 中寻找“good first issue”标签的任务，或参与 Discord 社区讨论，了解项目路线图和待开发功能。项目提供了多语言文档（含简体中文），上手门槛不高。

数据来源: github.com/trending

报告日期: 2026-08-20

由 DeepSeek AI 提供智能分析 | 自动生成